GEUVADIS Gene Expression GREML Workflow Handoff

Last updated: 2026−05−26

This document is a knowledge−transfer guide for the GEUVADIS gene expression heritability workflow in this repository. It is written for the next person who needs to understand, rerun, audit, or extend the analysis after the original workflow owner leaves the project.

The workflow estimates gene−level SNP heritability of RNA expression phenotypes using GEUVADIS RNA−seq data, 1000 Genomes genotype data, GTEx−inspired expression processing, and GCTA GREML. The repository also contains downstream diagnostic analyses to compare expression choices, normalization choices, SNP sets, and expression distribution properties.

Important upstream relationship:

```
The GREML workflow starts where the GEUVADIS Salmon workflow ends.
```

It does not start from FASTQ files. It starts from the two gene−level matrices created by the Salmon Nextflow handoff workflow:

```
gene_counts.tsv
gene_tpm.tsv
```

Everything downstream depends on those matrices being produced from the same GRCh37.75/hg19 reference, with compatible GEUVADIS sample naming, and with count and TPM matrices representing the same samples and genes.

Main handoff products:

```
results/summary/final_heritability_summary_<runLabel>.tsv
results/data/pheno_<runLabel>.phenotypes.tsv
results/data/pheno_<runLabel>.qcovar
results/data/pheno_<runLabel>.gene_index_map.txt
results/pca/<runLabel>/genotype_grm.*
results/pca/<runLabel>/geno_pca.eigenvec
runs/_analysis/*/plots/*.png
```

The summary TSV files are the primary scientific outputs. The phenotype, qcovar, gene map, GRM, and PCA files are the primary audit/provenance outputs. The ‘_analysis‘ plots are diagnostic and interpretive products built from the published run outputs.

Important source files for this handoff:

```
docs/greml_workflow_knowledge_transfer.md
docs/greml_workflow_knowledge_transfer.pdf
scripts/render_greml_workflow_knowledge_transfer_pdf.R
```

The markdown file is the source of truth. The PDF is the shareable handoff artifact. Regenerate the PDF after editing the markdown.

Regenerate command from the local repository checkout:

```
cd /Users/sl8085/Documents/MostafaviLab/Git/GeneExpression
Rscript scripts/render_greml_workflow_knowledge_transfer_pdf.R
```

If You Only Read One Page

Scientific question:

* Estimate how much gene expression variation across GEUVADIS European individuals is captured by genotype−derived relatedness.
* Compare how estimates change across SNP set, expression source, and phenotype normalization.

Data sources:

* RNA expression comes from the GEUVADIS Salmon workflow outputs: ‘gene_counts.tsv‘ and ‘gene_tpm.tsv‘.
* Genotypes come from the configured 1000 Genomes EUR PLINK prefix.
* Metadata comes from the GEUVADIS SDRF file.

Estimator:

* GCTA GREML estimates ‘h2_GREML = V(G) / Vp‘ per gene.
* The GRM defines genetic similarity among individuals.
* qcovars include genotype PCs and PEER factors.

Default 12 settings:

* SNP set: ‘all_snps‘ or ‘hm3_no_mhc‘.
* Expression source: ‘tmm‘ or ‘tpm‘.
* Normalization: ‘raw‘, ‘irnt‘, or ‘inverse_normal‘.

Main output:

```
runs/<run-name>/results/summary/final_heritability_summary_<runLabel>.tsv
```

Most important caveats:

* The default genotype prefix ends in ‘.22‘; verify whether this is chromosome 22 only. If yes, these are chromosome-22-tagged variance estimates, not genome-wide SNP heritability.
* ‘r_peer‘ was originally a personal conda environment. New users should test it or create their own.
* New users should set ‘NXF_WORK_ROOT=/gpfs/scratch/$USER/nextflow_work/greml_production‘.
* Full reuse ignores new sample filtering choices; do not combine ‘--reuse_run_dir‘ with a new unrelated keep file unless reused files already used that sample set.
* ‘raw‘ h2 and ‘irnt‘ h2 are different estimands because the phenotype scale changes.

First command for a new user:

```
cd /gpfs/data/mostafavilab/sool/analysis/GeneExpression/20260428_GE_GEUVADIS_v2/GeneExpression
mkdir -p /gpfs/scratch/$USER/nextflow_work/greml_production
NXF_WORK_ROOT=/gpfs/scratch/$USER/nextflow_work/greml_production GREML_FANOUT=0 sbatch run_greml.sh --us
e_hm3_no_hla false --expression_source tmm --normalization_type raw
```

Do the single-run smoke test before launching the 12-run fanout.

1. Scientific Question

The central question is:

How much of gene expression variation across European GEUVADIS individuals can be explained by genotype-derived relatedness, and how sensitive are those heritability estimates to expression quantification, normalization, SNP set, and expression distribution shape?

Operationally, each gene is treated as a quantitative phenotype. GCTA GREML is used to estimate the fraction of phenotypic variance explained by a genomic relationship matrix:

```
h2_GREML = V(G) / Vp
```

where ‘V(G)‘ is the additive genetic variance component estimated from the GRM and ‘Vp‘ is total phenotypic variance after the model accounts for covariates.

The workflow asks this question under multiple preprocessing choices:

* SNP set: all available SNPs in the genotype prefix versus HapMap3/no-MHC SNPs.
* Expression source: TPM-derived expression versus TMM-normalized count-derived expression.
* Phenotype normalization: raw expression scale versus inverse-normal transformed expression.
* Covariates: genotype PCs and PEER factors.

The broader scientific motivation is that expression heritability estimates can be sensitive to the phenotype scale. Low expression, zero inflation, skewed expression distributions, and different normalization choices can all change how GREML partitions variance.

2. Previous Research And What This Workflow Mimics

GEUVADIS and 1000 Genomes

GEUVADIS generated RNA−seq data for lymphoblastoid cell lines from 1000 Genomes individuals. The public GEUVADIS collection contains mRNA and small RNA sequencing from over 460 samples in CEU, FIN, GBR, TSI, and YRI. This workflow focuses on the European−ancestry groups used in many GEUVADIS analyses:

* British: GBR
* Finnish: FIN
* Tuscan: TSI
* Utah residents with Northern and Western European ancestry: CEU

The project uses GEUVADIS expression data because it has both population−scale RNA−seq and matched 1000 Genomes genotype data. That makes it a good test case for expression heritability: for the same individuals, we can construct gene expression phenotypes and genotype−derived GRMs.

Hernandez et al.−style expression heritability framing

Hernandez et al. studied gene expression heritability in GEUVADIS and emphasized how variant frequency classes, rare variants, and expression architecture affect estimated cis heritability. In their GEUVADIS European analysis, they started from European individuals and removed cryptically related samples using 1000 Genomes relatedness/identity−by−state information, ending with an unrelated EUR set.

This workflow mimics the Hernandez−style sample logic by:

* Starting from GEUVADIS EUR metadata.
* Mapping RNA−seq run IDs to 1000 Genomes genotype IDs.
* Optionally intersecting the mapped EUR sample set with ‘unrelated_keep_file‘.
* Using that sample set consistently for GRM, PCA, phenotype prep, and GREML.

Important distinction:

```
Hernandez−style pruning = external 1000G unrelated sample keep list
not necessarily GCTA −−grm−cutoff 0.025
```

The workflow now exposes this as:

```
params.unrelated_keep_file
```

GTEx−style expression phenotype processing

The workflow intentionally follows GTEx−like expression processing choices:

* Keep genes with TPM >= 0.1 in at least 20% of samples.
* Keep genes with counts >= 6 in at least 20% of samples.
* Use TMM normalization for counts.
* Use inverse−normal transformation per gene for normalized expression phenotypes.
* Use genotype PCs and PEER factors as covariates.
* Use a sample−size−based rule for PEER factors:
‘15, 30, 45, 60‘ for increasing sample−size bins.

GTEx used these types of steps to make expression phenotypes more robust for linear modeling and eQTL mapping. This workflow reuses that logic before GREML.

TMM normalization rationale

TMM, the trimmed mean of M−values method from Robinson and Oshlack, was developed because simple library−size scaling can be biased when RNA composition differs between samples. If a sample has a few highly expressed genes, those genes take up sequencing "real estate" and can make all other genes appear lower by simple proportional scaling. TMM estimates sample−specific scaling factors after trimming extreme log−fold changes and intensity values, producing effective library sizes for cross−sample comparison.

In this pipeline, when ‘expression_source=tmm‘, counts are converted to TMM−normalized CPM:

```
DGEList(counts)
calcNormFactors(method = "TMM")
cpm(normalized.lib.sizes = TRUE, log = FALSE, prior.count = 0)
```

GCTA GREML rationale

GCTA GREML uses a genomic relationship matrix as the covariance structure for random additive genetic effects. If genetically more similar individuals also have more similar expression for a gene, GREML attributes some expression variance to genotype. The output ‘V(G)/Vp‘ is interpreted as the SNP heritability captured by the SNPs used to build the GRM.

In this workflow, the GRM is built once per run setting and the same GRM is used across genes. Each gene is then selected from a multi−phenotype file using GCTA's ‘−−mpheno‘ option.

What We Mimic Versus What We Do Not Mimic

GEUVADIS data:

* We mimic prior GEUVADIS expression−genetics work by using GEUVADIS RNA−seq with matched 1000 Genomes genotypes.
* We do not reprocess all GEUVADIS study design metadata beyond the SDRF fields needed for run/sample/population mapping.

European−only analysis:

* We mimic the common EUR−only framing used to reduce continental ancestry heterogeneity.
* We do not claim the resulting h2 distributions generalize to YRI or other non−European populations.

Hernandez−style unrelated samples:

* We mimic the idea of removing cryptically related individuals by allowing an external 1000 Genomes/Hernandez−style unrelated keep list.
* We do not automatically reproduce Hernandez et al.'s exact keep set unless that exact keep file is supplied.
* We do not recommend blindly using GCTA ‘−−grm−cutoff 0.025‘ as a substitute without checking the resulting sample count.

GTEx−style expression filtering and PEER:

* We mimic GTEx−style expression filtering and sample−size−based PEER factor selection.
* GTEx optimized these choices mainly for eQTL discovery. In GREML, PEER can reduce unwanted variation but can also remove biological signal if over−specified.

GCTA GREML:

* We use GCTA GREML to estimate variance tagged by the chosen GRM.
* This is not identical to cis−only rare−variant heritability or total gene−expression heritability.
* ‘hm3_no_mhc‘ estimates are common, well−tagged, non−MHC SNP−tagged variance components.
* ‘all_snps‘ estimates depend on whatever variants are actually present in the configured PLINK prefix.

TPM/TMM/RAW/IRNT sensitivity:

* These are workflow sensitivity analyses designed to show how expression scale and normalization choices affect h2.

* A difference between settings should be interpreted as phenotype−definition sensitivity, not automatically as a biological discovery.

3. Repository Layout

Primary files:

```
README.md
run_greml.sh
nf/main.nf
nf/nextflow.config
nf/bin/prepare_phenotypes.R
scripts/
docs/
```

What each file or directory does:

* ‘README.md‘: repository−level orientation.
* ‘run_greml.sh‘: Slurm driver, fanout controller, run−directory isolation, launch locking, scratch setup, and Nextflow invocation.
* ‘nf/main.nf‘: DSL2 workflow defining EUR filtering, GRM/PCA creation, phenotype preparation, per−gene GREML, summary creation, and phenotype compression.
* ‘nf/nextflow.config‘: default input paths, scientific parameters, Slurm resources, retry settings, and process environments.
* ‘nf/bin/prepare_phenotypes.R‘: expression matrix loading, sample matching, GTEx−style filtering, TMM/TPM phenotype construction, RAW/IRNT transformation, PEER estimation, and qcovar/phenotype writing.
* ‘scripts/‘: downstream QC, plotting, decile, skewness, TPM/TMM comparison, cleanup, and archival helpers.
* ‘docs/‘: methods notes, run instructions, and transfer−of−knowledge documents.

Important documentation already in the repo:

```
docs/how_to_run_greml_nextflow.md
docs/how_to_prepare_gtex_like_phenotypes.md
docs/how_to_run_downstream_analysis.md
docs/downstream_analysis_methods.md
```

Important analysis scripts:

```
scripts/plot_greml_h2_summary_boxplot.R
scripts/pairwise_h2_comparisons.R
scripts/tmm_expression_h2_deciles.R
scripts/tmm_raw_skewness_analysis.R
scripts/tpm_tmm_peer_correlations.R
scripts/downstream_h2_regulatory_repeat_analysis.R
```

One−line purpose of the main analysis scripts:

* ‘scripts/plot_greml_h2_summary_boxplot.R‘: summarizes h2 distributions across run settings.
* ‘scripts/pairwise_h2_comparisons.R‘: compares gene−level h2 estimates between settings and colors by expression or Wald Z.
* ‘scripts/tmm_expression_h2_deciles.R‘: bins genes by expression abundance and compares h2 by decile.
* ‘scripts/tmm_raw_skewness_analysis.R‘: computes raw TMM skewness and plots example gene distributions.
* ‘scripts/tpm_tmm_peer_correlations.R‘: compares TPM−derived and TMM−derived phenotype values after processing.
* ‘scripts/downstream_h2_regulatory_repeat_analysis.R‘: older follow−up analysis linking h2 to constraint, regulatory annotations, and repeats.

4. Main HPC Paths

Project root on HPC:

```
/gpfs/data/mostafavilab/sool/analysis/GeneExpression/20260428_GE_GEUVADIS_v2/GeneExpression
```

Run directory root:

```
/gpfs/data/mostafavilab/sool/analysis/GeneExpression/20260428_GE_GEUVADIS_v2/GeneExpression/runs
```

Nextflow scratch/work root:

```
/gpfs/scratch/sl8085/nextflow_work/greml_production
```

RNA−seq matrices from the previous Salmon workflow:

```
/gpfs/data/mostafavilab/sool/analysis/GeneExpression/20260125_salmon_nextflow/results/matrices/gene_tpm.
tsv
/gpfs/data/mostafavilab/sool/analysis/GeneExpression/20260125_salmon_nextflow/results/matrices/gene_coun
ts.tsv
```

Default genotype prefix:

```
/gpfs/data/mostafavilab/shared_data/LDSC_data/1000G_EUR_Phase3_plink/1000G.EUR.QC.22
```

This resolves to:

```
1000G.EUR.QC.22.bed
1000G.EUR.QC.22.bim
1000G.EUR.QC.22.fam
```

The current prefix ends with '.22'. A successor should verify whether this is intentionally chromosome 22 only or whether the intended production GRM should be built from genome−wide merged chromosomes. This is one of the most important handoff checks.

GCTA executable:

```
/gpfs/data/mostafavilab/programs/GCTA/gcta−1.95.0−linux−kernel−3−x86_64/squashfs−root/AppRun
```

HM3/no−MHC SNP list:

```
/gpfs/data/mostafavilab/shared_data/LDSC_data/hm3_no_mhc_snps.txt
```

GEUVADIS SDRF metadata:

```
https://www.ebi.ac.uk/arrayexpress/files/E−GEUV−1/E−GEUV−1.sdrf.txt
```

Detailed input file inventory

This section is intentionally logistics−heavy. The goal is that the next person can identify each input on HPC, understand what biological or computational information it carries, and know why the workflow needs it.

Project repository root:

```
/gpfs/data/mostafavilab/sool/analysis/GeneExpression/20260428_GE_GEUVADIS_v2/GeneExpression
```

What it contains:

* 'run_greml.sh', the Slurm−facing driver script.
* 'nf/main.nf', the Nextflow workflow definition.
* 'nf/nextflow.config', the default file paths, parameters, resources, and Slurm profile.
* 'nf/bin/prepare_phenotypes.R', the R script that turns Salmon matrices plus metadata/covariates into GCTA−compatible phenotype and covariate files.
* 'scripts/', downstream plotting and QC scripts.
* 'docs/', handoff and methods documentation.

Why we use it:

* This is the reproducible analysis root. Submitting from this directory lets ‘run_greml.sh‘ resolve the intended Nextflow project files instead of accidentally using a Slurm spool path.
* The driver script checks the SHA−256 and modification time of ‘nf/bin/prepare_phenotypes.R‘ before launching. That protects against silently running a stale phenotype−preparation script.

Upstream GEUVADIS FASTQ input used by the Salmon workflow:

```
/gpfs/data/mostafavilab/reference_files/RNAseq/GEUVADIS/ERR*.fastq.gz
```

What it contains:

* GEUVADIS RNA−seq reads, one FASTQ per ENA run ID such as ‘ERR...‘.
* These are the raw sequencing inputs from which transcript and gene expression were quantified.

Why we use it:

* GEUVADIS provides RNA−seq from lymphoblastoid cell lines with matched 1000 Genomes genotypes.
* The ENA run IDs are later linked to individual IDs through the SDRF metadata, making it possible to match expression phenotypes to genotype samples.

GRCh37/hg19 transcriptome annotation used for Salmon quantification:

```
/gpfs/data/mostafavilab/reference_files/transcriptome/hg19/Homo_sapiens.GRCh37.75.gtf.gz
```

What it contains:

* Ensembl GRCh37 release 75 gene and transcript annotations.
* The upstream Salmon workflow parses ‘transcript_id‘ and ‘gene_id‘ from this GTF to create ‘tx2gene.tsv‘.

Why we use it:

* GEUVADIS and 1000 Genomes−era resources are commonly aligned to GRCh37/hg19.
* Using a matching GRCh37 transcriptome avoids mixing genome builds between RNA quantification and 1000 Genomes genotype resources.
* The GTF is required to aggregate transcript−level Salmon estimates to gene−level estimates.

GRCh37/hg19 cDNA fasta used for Salmon indexing:

```
/gpfs/data/mostafavilab/reference_files/transcriptome/hg19/Homo_sapiens.GRCh37.75.cdna.all.fa.gz
```

What it contains:

* Ensembl GRCh37 release 75 transcript sequences.
* Salmon uses this fasta to build the transcriptome index.

Why we use it:

* Salmon quantifies reads against transcript sequences. The cDNA fasta is the reference sequence input for the quasi−mapping index.
* It is paired with the GTF above, so transcript IDs in the fasta can be mapped back to gene IDs consistently.

Salmon TPM matrix:

```
/gpfs/data/mostafavilab/sool/analysis/GeneExpression/20260125_salmon_nextflow/results/matrices/gene_tpm.
tsv
```

What it contains:

* A gene−by−sample matrix produced by the previous Salmon workflow.
* First column is the gene ID; remaining columns are GEUVADIS run/sample columns.
* Values are Salmon gene−level TPM estimates.

Where it enters this workflow:

* ‘nf/nextflow.config‘ sets ‘params.tpm_file‘ to this path.
* ‘PREPARE_PHENOTYPES‘ passes it to ‘nf/bin/prepare_phenotypes.R‘.
* The R script reads it with ‘fread()‘, identifies the gene column, strips Ensembl version suffixes if present, maps sample columns to GEUVADIS run IDs, and applies the GTEx−style expression filter.

Why we use it:

* TPM normalizes within a sample for transcript/gene length and sequencing depth. It is useful for comparing relative expression abundance across genes within a sample.
* TPM is part of the expression filter even when the phenotype source is TMM. A gene must have TPM above threshold in enough samples and count support in enough samples.
* TPM can also be used directly as the phenotype source when ‘−−expression_source tpm‘.

Scientific caveat:

* TPM is compositional. If a few genes dominate a sample, TPM values for other genes can shift even when absolute molecule counts do not. This is one reason the workflow compares TPM−derived phenotypes to TMM−derived phenotypes.

Salmon estimated−count matrix:

```
/gpfs/data/mostafavilab/sool/analysis/GeneExpression/20260125_salmon_nextflow/results/matrices/gene_coun
ts.tsv
```

What it contains:

* A gene−by−sample matrix from Salmon ‘NumReads‘, aggregated to gene level.
* First column is gene ID; remaining columns correspond to GEUVADIS run/sample columns.
* Values are estimated counts, not necessarily integers, because Salmon probabilistically assigns ambiguous reads.

Where it enters this workflow:

* ‘nf/nextflow.config‘ sets ‘params.counts_file‘ to this path.
* ‘PREPARE_PHENOTYPES‘ passes it to ‘prepare_phenotypes.R‘.
* The R script uses counts for the GTEx−style count filter and for TMM normalization when ‘−−expression_source tmm‘.

Why we use it:

* TMM is defined on count data. It estimates sample−specific normalization factors that correct for library−size and composition effects.
* Counts preserve information about read support. A gene with TPM above threshold but very low count support can be unstable for heritability estimation, so both TPM and count filters are required.

GEUVADIS SDRF metadata:

```
https://www.ebi.ac.uk/arrayexpress/files/E−GEUV−1/E−GEUV−1.sdrf.txt
```

What it contains:

* Sample−level GEUVADIS metadata from ArrayExpress.
* Columns include RNA−seq run identifiers such as ‘ENA_RUN‘, ancestry/population labels, and individual/sample identifiers.

Where it enters this workflow:

* ‘FILTER_EUROPEANS‘ downloads it to ‘sdrf.txt‘ inside the Nextflow task work directory.
* The process detects the ancestry column and the ‘ENA_RUN‘ column.
* It keeps the configured European ancestry labels and writes ‘eur_map.tsv‘, which maps RNA run IDs to genotype FID/IID.

Why we use it:

* The expression matrices are indexed by RNA−seq run/sample columns, while the genotype files are indexed by 1000 Genomes FID/IID.
* The SDRF is the bridge between those two naming systems.
* It also allows the workflow to restrict to European GEUVADIS populations, reducing ancestry−driven structure in expression and genotype covariance.

European ancestry labels used from the SDRF:

```
British, Finnish, Tuscan, Utah
```

What they represent:

* British corresponds to GBR.
* Finnish corresponds to FIN.
* Tuscan corresponds to TSI.
* Utah corresponds to CEU.

Why we use them:

* These are the GEUVADIS European groups that match the 1000 Genomes EUR genotype resource.
* Restricting ancestry reduces confounding from major continental population structure. Residual European substructure is handled with genotype PCs.

1000 Genomes EUR PLINK prefix:

```
/gpfs/data/mostafavilab/shared_data/LDSC_data/1000G_EUR_Phase3_plink/1000G.EUR.QC.22
```

Files implied by the prefix:

```
1000G.EUR.QC.22.bed
1000G.EUR.QC.22.bim
1000G.EUR.QC.22.fam
```

What ‘.bed‘ contains:

* Binary PLINK genotype matrix.
* Rows/variants and individuals are encoded compactly for efficient genotype operations.

What ‘.bim‘ contains:

* Variant metadata: chromosome, SNP ID, genetic position, base−pair position, and alleles.
* GCTA uses this to know which variants are included and to apply optional SNP extraction.

What ‘.fam‘ contains:

* Sample metadata: family ID, individual ID, father ID, mother ID, sex, and phenotype placeholder.
* This workflow uses FID/IID from the FAM to match 1000 Genomes genotypes to GEUVADIS individuals.

Where it enters this workflow:

* ‘nf/main.nf‘ resolves ‘${params.plink_prefix}.bed‘, ‘.bim‘, and ‘.fam‘.
* ‘FILTER_EUROPEANS‘ reads the ‘.fam‘ file to keep only SDRF samples that exist in the genotype data.
* ‘GENERATE_PCA‘ passes the prefix to GCTA through ‘−−bfile‘.
* ‘PREPARE_PHENOTYPES‘ also receives the ‘.fam‘ file for sample−ID consistency.

Why we use it:

* GREML needs a GRM derived from genotype data for the same individuals whose expression is being modeled.
* The 1000 Genomes EUR genotype resource is the matched genotype backbone for GEUVADIS European expression samples.

Critical caveat:

* The configured prefix ends in ‘.22‘. A successor must verify whether this is intentionally chromosome 22 only or whether production should use a genome−wide merged 1000 Genomes EUR prefix.

* If this is chromosome 22 only, ‘h2_GREML‘ estimates are chromosome−22−tagged variance components, not genome−wide SNP heritability.

HapMap3/no−MHC SNP list:

```
/gpfs/data/mostafavilab/shared_data/LDSC_data/hm3_no_mhc_snps.txt
```

What it contains:

* A list of SNP IDs used with GCTA ‘−−extract‘.
* The intended variant set is HapMap3−style common, well−imputed, widely used SNPs excluding the MHC region.

Where it enters this workflow:

* When ‘−−use_hm3_no_hla true‘, ‘main.nf‘ validates this file and sets:

```
−−extract /gpfs/data/mostafavilab/shared_data/LDSC_data/hm3_no_mhc_snps.txt
```

* ‘GENERATE_PCA‘ passes that extraction argument to GCTA while building the GRM.

Why we use it:

* HapMap3 SNPs are a conservative common−SNP set often used in heritability and LDSC−style workflows.
* Excluding the MHC avoids a region with unusually long−range LD and strong immune−related effects that can dominate covariance patterns.
* Comparing ‘all_snps‘ against ‘hm3_no_mhc‘ tests how sensitive expression heritability estimates are to the SNP set used to build the GRM.

Optional unrelated keep file:

```
params.unrelated_keep_file = ""
```

Example command−line use:

```
−−unrelated_keep_file /path/to/geuvadis_1000g_unrelated_eur.keep
```

Expected content:

* Either one column containing IID, or two columns containing FID and IID.
* The workflow accepts both formats. If two columns are present, it first tries exact FID/IID matching and then falls back to IID matching if needed.

Where it enters this workflow:

* ‘FILTER_EUROPEANS‘ intersects the mapped GEUVADIS EUR samples with this keep list.
* The resulting ‘eur_keep.txt‘ is passed to GCTA ‘−−keep‘ during GRM/PCA construction.

Why we use it:

* Hernandez−style GEUVADIS analyses remove cryptically related samples before heritability analysis.
* The keep−list approach is different from blindly using GCTA ‘−−grm−cutoff 0.025‘. A keep list lets the user reproduce a specific unrelated panel from 1000 Genomes relationship metadata or PLINK KING/IBD pruning.

Caveat:

* If phenotype and GRM files are reused with ‘−−reuse_run_dir‘, the unrelated keep file does not change those reused artifacts. The reused files must already have been created from the same sample set.

GCTA executable:

```
/gpfs/data/mostafavilab/programs/GCTA/gcta−1.95.0−linux−kernel−3−x86_64/squashfs−root/AppRun
```

What it contains:

* The local GCTA 1.95 AppRun executable used for GRM construction, PCA, GREML, and optionally HE regression.

Where it enters this workflow:

* ‘GENERATE_PCA‘ calls it with ‘−−make−grm‘ and ‘−−pca‘.
* ‘ESTIMATE_HERITABILITY‘ calls it with ‘−−reml‘ for every gene.
* ‘run_he=false‘ is currently the default because HE regression hit a missing MKL runtime library in the AppRun environment.

Why we use it:

* GCTA GREML is the core variance−component estimator. It models covariance among individuals as a function of the genotype−derived GRM.
* Using one explicit executable path avoids version drift across login nodes or modules.

Phenotype−preparation script:

```
/gpfs/data/mostafavilab/sool/analysis/GeneExpression/20260428_GE_GEUVADIS_v2/GeneExpression/nf/bin/prepa
re_phenotypes.R
```

What it contains:

* The R implementation of sample matching, GTEx−style expression filtering, TMM calculation, raw/IRNT phenotype transformation, PEER factor estimation, covariate file generation, and GCTA phenotype formatting.

Where it enters this workflow:

* ‘PREPARE_PHENOTYPES‘ runs it through ‘Rscript‘.
* ‘run_greml.sh‘ and ‘main.nf‘ both validate that the script contains expected diagnostic markers.
* ‘main.nf‘ refuses to run if the selected script hash differs from the canonical ‘nf/bin‘ copy.

Why we use it:

* This is where most of the biological phenotype definition happens.
* The hash checks make the workflow safer after multiple debugging iterations because they prevent an older copy of the phenotype script from silently being used.

Conda/R runtime for phenotype preparation:

```
module load anaconda3/gpu/new
source activate r_peer
```

What it contains:

* R and required packages such as ‘data.table‘, ‘edgeR‘, and ‘peer‘.

Where it enters this workflow:

* The Slurm profile applies this ‘beforeScript‘ to ‘FILTER_EUROPEANS‘ and ‘PREPARE_PHENOTYPES‘.

Why we use it:

* ‘edgeR‘ is required for TMM normalization.
* ‘peer‘ is required to estimate PEER hidden factors.
* Keeping these in a named environment improves reproducibility and avoids relying on whatever R packages happen to be installed on a login node.

Important ownership note:

* The ‘r_peer‘ environment was created by the original workflow owner.
* A successor should not assume they can use it. On many HPC systems, a conda environment created inside another user's home or project space may be unreadable, non−writable, or tied to that user's shell initialization.
* Even if it is technically readable, relying on another user's personal environment is fragile because packages can be removed, paths can change, and permissions can break after account changes.

How to check whether the existing environment is accessible:

```
module load anaconda3/gpu/new
conda env list | grep −w r_peer || true
source activate r_peer
which R
Rscript −e 'pkgs <− c("data.table","edgeR","peer","matrixStats"); print(sapply(pkgs, requireNamespace, q
uietly=TRUE)); print(.libPaths()); sessionInfo()'
```

Interpretation:

* If ‘source activate r_peer‘ works and all required packages print ‘TRUE‘, the environment is usable from that account.
* If activation fails, packages are missing, or the library paths point to a location the new user cannot access reliably, create a new environment.

Recommended new−user environment creation:

```
module load anaconda3/gpu/new
conda create −n r_peer −c conda−forge −c bioconda \
  r−base \
  r−data.table \
  r−matrixstats \
  bioconductor−edger \
  peer
conda activate r_peer
Rscript −e 'library(data.table); library(edgeR); library(peer); library(matrixStats); sessionInfo()'
```

If the solver cannot find a compatible ‘peer‘ build, try the legacy Bioconda R package:

```
module load anaconda3/gpu/new
conda create −n r_peer_legacy −c conda−forge −c bioconda \
  r−base=3.4.1 \
  r−data.table \
  r−matrixstats \
  bioconductor−edger \
  r−peer
conda activate r_peer_legacy
Rscript −e 'library(data.table); library(edgeR); library(peer); library(matrixStats); sessionInfo()'
```

Why two options are shown:

* Bioconda lists a ‘peer‘ package and older ‘r−peer‘ builds.
* The important runtime test is whether R can load ‘library(peer)‘ together with ‘data.table‘, ‘edgeR‘, and ‘matrixStats‘.
* If neither conda route works on the HPC, ask the HPC support team or the lab to install PEER as a shared module or to provide a centrally managed conda environment.

Nextflow and Java runtime:

```
module load nextflow
```

What it contains:

* Nextflow and its Java runtime dependency on the HPC module system.

Where it enters this workflow:

* ‘run_greml.sh‘ loads Nextflow before launching ‘nf/main.nf‘.

Why we use it:

* Nextflow coordinates the many per−gene Slurm jobs, handles caching/resume, and publishes outputs into the run directories.

Run directory root:

```
/gpfs/data/mostafavilab/sool/analysis/GeneExpression/20260428_GE_GEUVADIS_v2/GeneExpression/runs
```

What it contains:

* One subdirectory per parameter combination, for example:

```
all_snps_tmm_raw_peerauto_pmg0_npc5
hm3_no_mhc_tpm_irnt_peerauto_pmg0_npc5
```

Where it enters this workflow:

* ‘run_greml.sh‘ sets each Nextflow launch directory to ‘runs/<combo−label>‘.
* ‘params.outdir‘ becomes ‘runs/<combo−label>/results‘.

Why we use it:

* Separate run directories prevent different SNP/expression/normalization combinations from overwriting each other.
* Keeping logs and results together makes it much easier to debug a specific parameter combination.

Scratch/work directory root:

```
/gpfs/scratch/sl8085/nextflow_work/greml_production
```

What it contains:

* Nextflow task work directories.
* Per−gene ‘.command.sh‘, ‘.command.run‘, ‘.command.out‘, ‘.command.err‘, staged input files, and temporary GCTA outputs.

Where it enters this workflow:

* ‘run_greml.sh‘ exports ‘NXF_WORK‘.
* ‘nextflow −w‘ points to the run−specific scratch directory.

Why we use it:

* Per−gene GREML produces many temporary files. Scratch storage is more appropriate than the lab project directory for high−volume Nextflow work.
* When debugging, this is where the exact per−task GCTA command and GCTA log can be found.

Important ownership note:

* The default path shown above is the original owner's scratch path.
* A successor should use their own scratch allocation, for example:

```
/gpfs/scratch/<new_user>/nextflow_work/greml_production
```

Why each user should use their own scratch path:

* Scratch directories are usually user−owned and may not be writable by other users.
* Nextflow creates thousands of hashed task directories for per−gene GREML jobs. Keeping those in another user's scratch area can cause permission errors, quota confusion, and cleanup problems.
* The lab project directory may have file−count or storage limits, and we already saw publish/file−volume issues when too many diagnostic files were written there.
* Scratch is the right place for high−I/O, restartable, temporary work. The durable outputs are copied to ‘runs/<run−name>/results‘.

How to set a new scratch path at submission time:

```
mkdir −p /gpfs/scratch/$USER/nextflow_work/greml_production
NXF_WORK_ROOT=/gpfs/scratch/$USER/nextflow_work/greml_production sbatch run_greml.sh
```

For a single explicit run:

```
mkdir −p /gpfs/scratch/$USER/nextflow_work/greml_production
NXF_WORK_ROOT=/gpfs/scratch/$USER/nextflow_work/greml_production \
  sbatch run_greml.sh \
  −−use_hm3_no_hla false \
  −−expression_source tmm \
  −−normalization_type raw
```

What this changes:

* ‘run_greml.sh‘ sets ‘NXF_WORK=${NXF_WORK_ROOT}/${RUN_KEY}‘.
* Nextflow then runs each task under the new user's scratch directory.
* Published results still go to the run directory under ‘GREML_RUN_DIR‘ or the submit directory.

Reusable phenotype inputs from prior runs:

```
runs/<run-name>/results/data/pheno_<snp>_<source>_<normalization>.phenotypes.tsv
runs/<run-name>/results/data/pheno_<snp>_<source>_<normalization>.qcovar
runs/<run-name>/results/data/pheno_<snp>_<source>_<normalization>.gene_index_map.txt
runs/<run-name>/results/data/pheno_<snp>_<source>_<normalization>.filtered_gene_ids.txt
```

What the phenotype file contains:

* GCTA-compatible phenotype matrix with no header.
* First two columns are FID and IID.
* Each remaining column is one gene phenotype.

What the qcovar file contains:

* GCTA-compatible quantitative covariate matrix with no header.
* First two columns are FID and IID.
* Remaining columns are genotype PCs and PEER factors.

What the gene index map contains:

* Headered table linking ‘gene_name‘ to ‘mpheno_index‘.
* GCTA uses ‘--mpheno <index>‘ to choose the gene column in the phenotype matrix.

What the filtered gene list contains:

* Genes that passed the mandatory TPM/count expression filter before phenotype transformation.

Where they enter this workflow:

* If ‘--reuse_pheno_dir‘ or ‘--reuse_run_dir‘ is set, ‘main.nf‘ can skip ‘PREPARE_PHENOTYPES‘ and feed these files directly into ‘ESTIMATE_HERITABILITY‘.

Why we use them:

* Preparing phenotypes can take many hours because PEER is fit over a large gene-by-sample matrix.
* Reusing these files is appropriate when only the GCTA/GREML part changed and the sample set, expression source, normalization, PCs, and PEER settings are unchanged.

Reusable GRM/PCA inputs from prior runs:

```
runs/<run-name>/results/pca/<snp>_<source>_<normalization>/genotype_grm.grm.bin
runs/<run-name>/results/pca/<snp>_<source>_<normalization>/genotype_grm.grm.N.bin
runs/<run-name>/results/pca/<snp>_<source>_<normalization>/genotype_grm.grm.id
runs/<run-name>/results/pca/<snp>_<source>_<normalization>/geno_pca.eigenvec
```

What they contain:

* ‘.grm.bin‘ contains the binary lower-triangle GRM values.
* ‘.grm.N.bin‘ contains the number of SNPs used for each pairwise relationship estimate.
* ‘.grm.id‘ contains the FID/IID order of individuals in the GRM.
* ‘geno_pca.eigenvec‘ contains genotype principal components for the kept samples.

Where they enter this workflow:

* If ‘--reuse_grm_dir‘ or ‘--reuse_run_dir‘ is set, ‘main.nf‘ can skip ‘FILTER_EUROPEANS‘ and ‘GENERATE_PCA‘ and stage the existing ‘genotype_grm*‘ files into GREML tasks.

Why we use them:

* GRM/PCA files depend only on genotype prefix, SNP set, and kept samples. They do not depend on TPM versus TMM or raw versus IRNT expression.

* Reusing them saves time and avoids needless re−creation of identical genotype artifacts.

Downstream summary input files:

```
runs/<run−name>/results/summary/final_heritability_summary_<snp>_<source>_<normalization>.tsv
```

What they contain:

* One row per gene.
* Columns include ‘Gene‘, ‘Status‘, ‘Index‘, ‘h2_GREML‘, ‘SE_GREML‘, ‘Pval_GREML‘, ‘h2_HE‘, and ‘SE_HE‘.

Where they enter downstream analyses:

* ‘scripts/plot_greml_h2_summary_boxplot.R‘ reads them for global h2 distributions.
* ‘scripts/pairwise_h2_comparisons.R‘ reads them for pairwise h2 scatter plots.
* ‘scripts/tmm_expression_h2_deciles.R‘ reads them for expression decile analyses.
* ‘scripts/tmm_raw_skewness_analysis.R‘ reads them for skewness versus h2 analyses.

Why we use them:

* They are the canonical per−run GREML result table.
* Plotting scripts should use these summary files rather than scratch work directories because the summary files are stable, published outputs.

Downstream phenotype inputs used by plotting/QC scripts:

```
runs/<run−name>/results/data/pheno_<snp>_<source>_<normalization>.phenotypes.tsv
runs/<run−name>/results/data/pheno_<snp>_<source>_<normalization>.gene_index_map.txt
```

What they contain:

* The phenotype matrix contains sample−level expression phenotypes after the selected expression source, normalization mode, and PEER/PCA covariate preparation.
* The gene index map maps phenotype columns back to Ensembl gene IDs.

Where they enter downstream analyses:

* ‘scripts/tmm_expression_h2_deciles.R‘ reads RAW TMM and RAW TPM phenotype files to compute expression means, medians, deciles, and skewness bins.
* ‘scripts/tmm_raw_skewness_analysis.R‘ reads RAW TMM phenotype files to compute skewness per gene and selected expression−distribution examples.
* ‘scripts/tpm_tmm_peer_correlations.R‘ reads TPM and TMM phenotype files to compare post−processing expression values gene−by−gene.
* ‘scripts/pairwise_h2_comparisons.R‘ reads RAW expression phenotype files to color h2 scatter plots by log2 mean expression.

Why we use them:

* Summary files contain h2 but not the underlying expression distribution.
* Phenotype files are needed to ask whether h2 differences are associated with expression abundance, zero inflation, skewness, or TPM/TMM phenotype disagreement.

Caveat:

* The phenotype files are already transformed according to the run setting. For expression−distribution questions, use RAW TMM or RAW TPM runs rather than IRNT runs, because IRNT intentionally erases the native expression scale.

Analysis output root on HPC:

```
/gpfs/data/mostafavilab/sool/analysis/GeneExpression/20260428_GE_GEUVADIS_v2/GeneExpression/runs/_analys
is
```

What it contains:

* Organized downstream tables and plots produced by the R scripts in ‘scripts/‘.

* Subdirectories include ‘plot_greml_h2_summary_boxplot‘, ‘pairwise_h2_comparisons‘, ‘tmm_expression_h2_deciles‘, ‘tmm_raw_skewness‘, and ‘tpm_tmm_peer_correlations‘.

Why we use it:

* It separates exploratory/diagnostic plots from per−run Nextflow outputs.
* It makes it easier to copy or archive analysis products without touching Nextflow cache or scratch directories.

Slurm and Nextflow log locations:

```
runs/nextflow_logs/nextflow_driver−<combo_label>−<jobid>.out
runs/nextflow_logs/nextflow_driver−<combo_label>−<jobid>.err
runs/<run−name>/nextflow_logs/nextflow_<jobid>.log
```

What they contain:

* Driver stdout/stderr from the Slurm job that launches Nextflow.
* Nextflow engine logs for a specific run session.
* Launch information, resolved paths, resume status, run key, work directory, parameter values, task failures, and work−directory pointers.

Why we use them:

* These are the first place to look when a run fails before producing a final summary.
* They provide the exact work directory for failed tasks, which can then be inspected through ‘.command.sh‘, ‘.command.err‘, ‘.command.out‘, ‘.exitcode‘, and GCTA logs.

Observed production values from previous logs:

```
mapped/sample count in representative GCTA runs: 344 individuals
gene traits in representative phenotype files: 16690
GCTA quantitative covariates reported in representative runs: 49
fixed effects reported by GCTA: 50 including intercept
```

Interpretation:

* These are observed examples from previous production/debug logs, not hard−coded guarantees.
* A successor should verify them per run because unrelated sample pruning, genotype prefix changes, or phenotype filtering changes can alter the numbers.
* With ‘N‘ around 344, the GTEx−style rule requests 45 PEER factors. The observed ‘49‘ GCTA quantitative covariates are consistent with 5 genotype PCs plus approximately 44 PEER−derived covariate columns after PEER output handling. Always verify the actual qcovar column count.

Quick verification commands:

```
RUN_DIR=runs/all_snps_tmm_raw_peerauto_pmg0_npc5
wc −l "$RUN_DIR/results/pca/all_snps_tmm_raw/genotype_grm.grm.id"
awk −F'\t' 'NR==1 {print NF−2}' "$RUN_DIR/results/data/pheno_all_snps_tmm_raw.phenotypes.tsv"
awk −F'\t' 'NR==1 {print NF−2}' "$RUN_DIR/results/data/pheno_all_snps_tmm_raw.qcovar"
```

Run−specific scratch work directories:

```
/gpfs/scratch/sl8085/nextflow_work/greml_production/<run−name>
```

What they contain:

* Hashed Nextflow task directories for ‘FILTER_EUROPEANS‘, ‘GENERATE_PCA‘, ‘PREPARE_PHENOTYPES‘, ‘ESTIMATE_HERITABILITY‘, ‘SUMMARIZE_RESULTS‘, and ‘COMPRESS_PHENOTYPE‘.
* Per−gene intermediate GCTA files such as ‘<gene>_greml.gcta.log‘, ‘<gene>_greml.hsq‘, ‘<gene>.stats‘, and ‘.command.*‘ wrapper files.

Why we use them:

* Published results are clean and compact, while scratch work directories preserve the exact command−level evidence needed for debugging.
* If a gene fails or is marked ‘FAIL‘, the scratch task directory is where to inspect the GCTA reason.

5. Upstream RNA−seq Quantification

The previous Nextflow workflow quantified GEUVADIS RNA−seq with Salmon using GRCh37/hg19 Ensembl 75 transcriptome references:

```
Homo_sapiens.GRCh37.75.gtf.gz
Homo_sapiens.GRCh37.75.cdna.all.fa.gz
```

It did the following:

1. Build a Salmon transcriptome index from the cDNA fasta.
2. Build a ‘tx2gene.tsv‘ map from the GTF.
3. Run ‘salmon quant‘ per GEUVADIS FASTQ in single−end mode with automatic library−type detection and gene−level aggregation:

```
salmon quant
−l A
−r <FASTQ>
−−geneMap tx2gene.tsv
```

Current upstream caveat:

```
The current GEUVADIS_Salmon pipeline has params.extra_quant = "".
```

That means the current reproducible upstream Salmon run does not add ‘−−validateMappings‘, ‘−−seqBias‘, or ‘−−gcBias‘. If a future analyst adds those flags, they should treat the resulting matrices as a new expression quantification version and rerun all downstream GREML phenotypes rather than mixing them with the current matrices.

4. Merge per−sample ‘quant.genes.sf‘ into:

```
gene_counts.tsv
gene_tpm.tsv
```

The Salmon workflow itself produced TPM and estimated counts. It did not create TMM. TMM is calculated later during phenotype preparation from the gene−level count matrix.

6. Main Nextflow Workflow Logic

The main workflow is in:

```
nf/main.nf
```

The main configuration is in:

```
nf/nextflow.config
```

The Slurm driver is:

```
run_greml.sh
```

GCTA input contract

At the moment GCTA runs, each gene−level GREML task needs exactly three scientific inputs plus one gene index:

```
genotype_grm.*
pheno_<runLabel>.phenotypes.tsv
pheno_<runLabel>.qcovar
pheno_<runLabel>.gene_index_map.txt
```

The GRM defines genetic similarity among the kept individuals. The phenotype file contains one expression phenotype column per gene, with no header because GCTA expects FID/IID followed by phenotype columns. The qcovar file contains the same FID/IID sample order plus quantitative covariates, mainly genotype PCs and PEER factors. The gene index map is the human−readable bridge between an Ensembl gene ID and the ‘−−mpheno‘ column index used by GCTA.

The biological requirement is that all three sample−indexed inputs must describe the same individuals:

```
GRM sample order
phenotype FID/IID rows
qcovar FID/IID rows
```

If these disagree, GCTA can fail outright or estimate h2 on the wrong sample intersection. This is why sample filtering, unrelated−sample pruning, PCA, PEER, phenotype creation, and GRM creation must be treated as one coherent input bundle.

Step 1: Slurm driver and fanout

The usual entry point is:

```
sbatch run_greml.sh
```

If no CLI arguments are supplied, ‘run_greml.sh‘ defaults to fanout mode and submits 12 child jobs:

```
2 SNP sets x 2 expression sources x 3 normalization modes = 12 runs
```

Default fanout dimensions:

```
SNP sets: hm3_no_mhc, all_snps
Expression sources: tpm, tmm
Normalization types: irnt, inverse_normal, raw
```

Current child run labels look like:

```
all_snps_tmm_raw_peerauto_pmg0_npc5
all_snps_tmm_irnt_peerauto_pmg0_npc5
hm3_no_mhc_tpm_inverse_normal_peerauto_pmg0_npc5
```

The script prints a combo hash, but the current run directory name does not include that hash.

Step 2: FILTER_EUROPEANS

This process:

1. Reads GEUVADIS SDRF metadata.
2. Detects the population column by searching for ‘ancestry category‘.
3. Detects the RNA run column by searching for ‘ENA_RUN‘.
4. Keeps European groups:

```
British
Finnish
Tuscan
Utah
```

5. Maps GEUVADIS samples to PLINK ‘.fam‘ IIDs.
6. Optionally intersects with ‘unrelated_keep_file‘.
7. Emits:

```
eur_keep.txt
eur_map.tsv
```

‘eur_keep.txt‘ is the FID/IID keep list used by GCTA. ‘eur_map.tsv‘ maps RNA run IDs to genotype FID/IID and is used by phenotype preparation.

Step 3: GENERATE_PCA

This process builds the GRM:

```
gcta --bfile <plink prefix> \
  --keep eur_keep.txt \
  <optional --extract hm3_no_mhc_snps.txt> \
  --make-grm \
  --out genotype_grm \
  --thread-num <cpus>
```

Then it runs PCA from the GRM:

```
gcta --grm genotype_grm \
  --pca 5 \
  --out geno_pca \
  --thread-num <cpus>
```

Outputs are published to:

```
results/pca/<runLabel>/
```

Key outputs:

```
geno_pca.eigenvec
genotype_grm.grm.bin
genotype_grm.grm.N.bin
genotype_grm.grm.id
genotype_grm.log
```

Scientific rationale:

* The GRM defines expected genetic covariance among individuals.
* PCA captures broad ancestry structure and is included as a fixed covariate.
* HM3/no−MHC reduces complexity from high−LD and difficult regions and improves comparability with common SNP reference analyses.

How to think about GRM values:

* A GRM entry is a genotype−derived estimate of how genetically similar two individuals are relative to the sample/reference allele frequencies.
* Values near 0 mean two unrelated individuals have about average similarity.
* Positive values mean two individuals are more genetically similar than average.
* Very high positive values suggest close relatives or duplicated/near−duplicated samples.

Approximate relationship scale:

```
parent/child or full siblings: about 0.5
grandparent/grandchild, half siblings, avuncular: about 0.25
first cousins: about 0.125
second cousins: about 0.03125
third cousins: about 0.0078
```

Why the GCTA ‘0.025‘ cutoff matters:

* A cutoff near ‘0.025‘ roughly removes relationships stronger than about second−cousin scale.
* This can be appropriate for phenotype heritability analyses that require unrelated individuals.
* In this GEUVADIS expression workflow, applying ‘−−grm−cutoff 0.025‘ blindly was too aggressive in one check and left only a tiny sample set.
* The preferred Hernandez−style approach here is to provide an external unrelated keep list through ‘−−unrelated_keep_file‘, then verify the final sample count.

Tiny GRM intuition example:

```
SNP1 standardized genotype: individual A = +1, individual B = +1
SNP2 standardized genotype: individual A = −1, individual B = −1
SNP3 standardized genotype: individual A = +1, individual B = −1
average product = (1*1 + −1*−1 + 1*−1) / 3 = 0.33
```

The positive average means A and B share more genotype state than average across these toy SNPs. Real GRMs average this idea across many variants with allele frequency standardization.

Step 4: PREPARE_PHENOTYPES

This process runs:

```
nf/bin/prepare_phenotypes.R
```

It receives:

```
TPM matrix
count matrix
eur_map.tsv
geno_pca.eigenvec
PLINK .fam
phenotype prefix
PEER settings
GTEx-style filter thresholds
expression source
normalization type
```

The script performs:

1. Sample mapping and ordering.
2. Gene ID cleanup.
3. Duplicate gene handling.
4. Mandatory expression filter.
5. TMM or TPM phenotype-source selection.
6. RAW or inverse-normal phenotype normalization.
7. PEER factor estimation.
8. PCA and PEER covariate merge.
9. GCTA-compatible phenotype and qcovar file writing.

Outputs are published to:

```
results/data/
```

Important files:

```
pheno_<runLabel>.phenotypes.tsv
pheno_<runLabel>.qcovar
pheno_<runLabel>.gene_index_map.txt
pheno_<runLabel>.filtered_gene_ids.txt
pheno_<runLabel>.phenotypes.tsv.gz
```

The phenotype and qcovar files intentionally have no header because GCTA expects that format. The gene index map has a header.

Step 5: ESTIMATE_HERITABILITY

This process runs once per gene. The workflow reads the gene index map, then submits one ‘ESTIMATE_HERITABILITY‘ task per gene.

GREML command:

```
gcta --grm genotype_grm \
  --pheno pheno_<runLabel>.phenotypes.tsv \
  --mpheno <gene index> \
  --qcovar pheno_<runLabel>.qcovar \
  --reml \
  --out <gene>_greml \
  --thread-num 1
```

The output ‘.hsq‘ file contains:

```
V(G)
V(e)
Vp
V(G)/Vp
Pval
```

The pipeline extracts:

```
h2_GREML
SE_GREML
Pval_GREML
```

HE regression is implemented but disabled by default:

```
run_he = false
```

This was disabled because earlier HE runs failed due an Intel MKL shared−library issue in the GCTA AppRun environment. GREML is the primary output.

Step 6: SUMMARIZE_RESULTS

This process concatenates one ‘.stats‘ file per gene into:

```
results/summary/final_heritability_summary_<runLabel>.tsv
```

Columns:

```
Gene
Status
Index
h2_GREML
SE_GREML
Pval_GREML
h2_HE
SE_HE
```

Each gene gets ‘PASS‘ or ‘FAIL‘. Per−gene failures do not necessarily fail the whole workflow because the workflow writes a ‘.stats‘ row and exits the gene job cleanly.

7. Parameter Table And Scientific Meaning

Sample and genotype parameters

```
eur_pops = ["British", "Finnish", "Tuscan", "Utah"]
```

Meaning:

* Keeps the four European GEUVADIS populations.
* Excludes YRI because the primary analysis is EUR−only.
* Mimics GEUVADIS/Hernandez−style European analysis.

```
unrelated_keep_file = ""
```

Meaning:

* Optional one−column IID or two−column FID/IID file.
* When supplied, it restricts mapped GEUVADIS EUR samples to an external unrelated sample set.
* This is the preferred way to mimic Hernandez−style cryptic−relatedness pruning.

```
use_hm3_no_hla = true
```

Meaning:

* Uses HapMap3 SNPs excluding the MHC/HLA region when true.
* Rationale: HM3 SNPs are a common well−imputed/common−variant reference set; excluding MHC reduces extreme LD and complex−region behavior.

```
n_pcs = 5
```

Meaning:

* Uses the top five genotype PCs as covariates.
* Rationale: genotype PCs capture broad ancestry structure even within EUR.
* This is slightly different from GTEx V7, which listed top three genotype PCs, but five PCs is a conservative choice for a smaller population−genetic analysis.

Expression filtering parameters

```
gtex_tpm_threshold = 0.1
gtex_count_threshold = 6
gtex_sample_frac_threshold = 0.2
```

Meaning:

* A gene must have TPM >= 0.1 in at least 20% of samples.
* The same gene must also have count >= 6 in at least 20% of samples.
* Both conditions must be true.

Scientific rationale:

* TPM threshold removes near−background transcription.
* Count threshold removes genes without enough read evidence.
* The 20% rule prevents a gene expressed in only a few outlier individuals from entering the GREML analysis.
* This follows GTEx−style eQTL expression filtering.

Expression−source parameters

```
expression_source = "tmm"
```

Allowed:

```
tmm
tpm
```

If ‘tmm‘, the pipeline starts from counts, calculates TMM factors with edgeR, and uses TMM−normalized CPM.

If ‘tpm‘, the pipeline uses Salmon−derived TPM directly.

Scientific difference:

* TPM corrects within sample for gene/transcript length and sequencing depth.
* TMM uses raw count composition across samples to estimate effective library size scaling factors.
* TPM is useful for abundance; TMM is often preferred for cross−sample count comparisons because it addresses RNA composition bias.

Simple TPM intuition:

```
TPM asks: within this sample, after accounting for gene length, what fraction of
the transcript pool belongs to each gene?
```

This makes TPM useful for abundance, but because all TPM values in a sample sum to a constant, one extremely high gene can make other genes look proportionally smaller.

Simple TMM intuition:

```
Sample A: most genes have ordinary counts.
Sample B: one or a few genes are extremely highly expressed.
```

If we normalized only by total library size, Sample B's effective denominator would be inflated by those extreme genes, making many ordinary genes look lower in Sample B. TMM trims extreme log−fold changes and intensity values, estimates a scaling factor, and uses that factor to create a more comparable effective library size. In this workflow, the actual phenotype is TMM−normalized CPM.

Normalization parameters

```
normalization_type = "irnt"
```

Allowed:

```
raw
irnt
inverse_normal
```

‘raw‘:

* Uses TMM CPM or TPM values directly.
* Most biologically interpretable scale.
* More vulnerable to zero inflation and skewness.

‘irnt‘:

* Uses ‘log2(x + 1)‘.
* Applies per−gene inverse−normal transform with Blom offset ‘3/8‘.
* Makes each gene approximately standard−normal across samples.
* Less interpretable in expression units but better aligned with linear−model assumptions.

Simple IRNT intuition:

```
raw expression values: 0, 0, 1, 3, 50
rank order:            1, 2, 3, 4, 5
IRNT output:           normal quantiles based on those ranks
```

The extreme value ‘50‘ no longer dominates by magnitude; it only contributes as the highest rank. This improves robustness to skewness and outliers, but the heritability estimate is now for rank−normalized expression rather than native TPM or TMM CPM units.

‘inverse_normal‘:

* Similar to ‘irnt‘, but uses rank offset ‘0.5‘.
* Kept mainly for comparison.

PEER parameters

```
peer_nk = "auto"
peer_max_genes = 0
```

GTEx−style auto rule:

```
N < 150: 15 PEER factors
150 <= N < 250: 30 PEER factors
250 <= N < 350: 45 PEER factors
N >= 350: 60 PEER factors
```

Scientific rationale:

* PEER estimates hidden factors that capture technical and biological unwanted variation, such as batch effects or broad latent expression patterns.
* GTEx chose the sample−size rule based on optimizing eGene discovery.
* ‘peer_max_genes = 0‘ means use all filtered genes for PEER.
* Setting ‘peer_max_genes > 0‘ uses top−variance genes only and is a speed option.

How PEER enters this workflow:

* PEER is fit from the sample−by−gene expression matrix after expression−source selection and normalization.
* The estimated PEER factors are not the phenotype. They are added to the qcovar file as fixed−effect covariates.
* GCTA then estimates h2 after adjusting for those PEER factors and genotype PCs.

What PEER can capture:

* RNA quality differences.
* Batch effects.
* Cell−state differences.
* Broad latent expression programs.
* Other unmeasured technical or biological effects shared across many genes.

Important caveat:

* PEER can also remove real biological signal if too many factors are included.
* The GTEx rule is defensible and widely used, but it was optimized for eQTL discovery rather than specifically for GREML heritability.
* With about 344 samples, the rule requests 45 PEER factors. Previous GCTA logs reported 49 quantitative covariates total, consistent with 5 PCs plus roughly 44 PEER−derived columns after PEER output handling. Verify actual qcovar columns per run.

GCTA parameters

```
run_he = false
```

Meaning:

* Only GREML is run by default.
* Haseman−Elston regression is optional and currently skipped.

```
gcta_path = /gpfs/data/mostafavilab/programs/GCTA/...
```

Meaning:

* Exact GCTA executable used by the workflow.

Slurm resources and retry behavior

Driver job from ‘run_greml.sh‘:

```
job name = NF_GREML
partition = mostafavilab
cpus = 1
memory = 8 GB
time = 7 days
```

Process resources from ‘nf/nextflow.config‘:

```
FILTER_EUROPEANS:
  cpus = 1
  memory = 8 GB
  time = 1h
  environment = r_peer

GENERATE_PCA:
  cpus = 4
  memory = 16 GB
  time = 2h

PREPARE_PHENOTYPES:
  cpus = 4
  memory = 64 GB
  time = 24h
  environment = r_peer

ESTIMATE_HERITABILITY:
  cpus = 1
  memory = 8 GB * attempt
  time = 4h * attempt
  max retries = 2
  cluster option = --signal=TERM@120 --requeue

SUMMARIZE_RESULTS:
  cpus = 1
  memory = 16 GB
  time = 2h
```

Retry behavior:

* ‘ESTIMATE_HERITABILITY‘ retries common external−kill statuses such as memory/time/signal−related exits.
* Per−gene GCTA failures are converted into one−line ‘FAIL‘ stats so a few bad genes do not necessarily abort the entire run.
* A completed Nextflow run can still contain failed genes. Always inspect ‘Status‘ in the summary.

8. Operational Preflight Checklist

Most workflow failures are operational rather than statistical: wrong path, missing module, personal conda environment, scratch permissions, stale reuse, or GCTA runtime issues. Run this before submitting the 12−run fanout.

Be in the project root:

```
cd /gpfs/data/mostafavilab/sool/analysis/GeneExpression/20260428_GE_GEUVADIS_v2/GeneExpression
test −f run_greml.sh
test −f nf/main.nf
test −f nf/nextflow.config
test −f nf/bin/prepare_phenotypes.R
```

Check Nextflow and Java:

```
module load nextflow
nextflow −version
java −version
```

Check the R/PEER environment:

```
module load anaconda3/gpu/new
source activate r_peer
Rscript −e 'pkgs <− c("data.table","edgeR","peer","matrixStats"); print(sapply(pkgs, requireNamespace, q
uietly=TRUE)); sessionInfo()'
```

Check required input files:

```
test −s /gpfs/data/mostafavilab/sool/analysis/GeneExpression/20260125_salmon_nextflow/results/matrices/g
ene_tpm.tsv
test −s /gpfs/data/mostafavilab/sool/analysis/GeneExpression/20260125_salmon_nextflow/results/matrices/g
ene_counts.tsv
test −s /gpfs/data/mostafavilab/shared_data/LDSC_data/1000G_EUR_Phase3_plink/1000G.EUR.QC.22.bed
test −s /gpfs/data/mostafavilab/shared_data/LDSC_data/1000G_EUR_Phase3_plink/1000G.EUR.QC.22.bim
test −s /gpfs/data/mostafavilab/shared_data/LDSC_data/1000G_EUR_Phase3_plink/1000G.EUR.QC.22.fam
test −s /gpfs/data/mostafavilab/shared_data/LDSC_data/hm3_no_mhc_snps.txt
test −x /gpfs/data/mostafavilab/programs/GCTA/gcta−1.95.0−linux−kernel−3−x86_64/squashfs−root/AppRun
```

Check the GEUVADIS SDRF is reachable:

```
curl −L −−head https://www.ebi.ac.uk/arrayexpress/files/E−GEUV−1/E−GEUV−1.sdrf.txt
```

Create user−owned scratch:

```
mkdir −p /gpfs/scratch/$USER/nextflow_work/greml_production
```

Minimum preflight interpretation:

* If ‘r_peer‘ fails, create a personal environment before running.
* If input ‘test −s‘ commands fail, fix paths before running.
* If ‘1000G.EUR.QC.22‘ is not intended to be chromosome 22 only, replace the genotype prefix before interpreting h2 as genome−wide.
* If scratch is not writable, set ‘NXF_WORK_ROOT‘ to a user−owned path.

9. How To Run The Workflow

Go to the project root on HPC:

```
cd /gpfs/data/mostafavilab/sool/analysis/GeneExpression/20260428_GE_GEUVADIS_v2/GeneExpression
```

Recommended first−time setup for a new user:

```
module load anaconda3/gpu/new
source activate r_peer
Rscript −e 'pkgs <− c("data.table","edgeR","peer","matrixStats"); print(sapply(pkgs, requireNamespace, q
uietly=TRUE))'
mkdir −p /gpfs/scratch/$USER/nextflow_work/greml_production
```

If ‘source activate r_peer‘ fails, create a personal replacement environment using the instructions in the input inventory section. Do not assume the original owner's conda environment will remain accessible after handoff.

Recommended submission pattern for a new user:

```
NXF_WORK_ROOT=/gpfs/scratch/$USER/nextflow_work/greml_production sbatch run_greml.sh
```

This keeps the durable outputs under the run directory while putting high−volume Nextflow task work under the new user's scratch allocation.

Current run naming behavior:

```
run_key = <snp_set>_<expression_source>_<normalization>_peer<peer_nk>_pmg<peer_max_genes>_npc<n_pcs>
runLabel = <snp_set>_<expression_source>_<normalization>
phenotype prefix = pheno_<runLabel>
summary filename = final_heritability_summary_<runLabel>.tsv
```

Example:

```
run_key = all_snps_tmm_raw_peerauto_pmg0_npc5
runLabel = all_snps_tmm_raw
phenotype = pheno_all_snps_tmm_raw.phenotypes.tsv
summary = final_heritability_summary_all_snps_tmm_raw.tsv
```

Important current behavior:

* Current ‘run_greml.sh‘ uses hash−free run directories by default, such as ‘runs/all_snps_tmm_raw_peerauto_pmg0_npc5‘.
* Older hash−suffixed folders may exist from earlier development iterations. Treat them as historical runs unless you intentionally point reuse parameters at them.

Default fanout combinations:

```
SNP sets: all_snps, hm3_no_mhc
Expression sources: tpm, tmm
Normalizations: irnt, inverse_normal, raw
Total runs: 2 x 2 x 3 = 12
```

Submit the default 12−run fanout:

```
sbatch run_greml.sh
```

Submit a single run:

```
GREML_FANOUT=0 sbatch run_greml.sh \
  --use_hm3_no_hla false \
  --expression_source tmm \
  --normalization_type raw
```

Run with Hernandez−style unrelated keep list:

```
GREML_FANOUT=0 sbatch run_greml.sh \
  --use_hm3_no_hla false \
  --expression_source tmm \
  --normalization_type raw \
  --unrelated_keep_file /path/to/geuvadis_1000g_unrelated_eur.keep
```

Submit 12 fanout runs with unrelated keep file:

```
GREML_FANOUT=1 GREML_FANOUT_INTERVAL_SEC=0 sbatch run_greml.sh \
  --unrelated_keep_file /path/to/geuvadis_1000g_unrelated_eur.keep
```

Disable resume:

```
GREML_RESUME=0 sbatch run_greml.sh ...
```

Use a different run root:

```
GREML_RUN_DIR=/path/to/output_root sbatch run_greml.sh ...
```

Use a different scratch root:

```
NXF_WORK_ROOT=/gpfs/scratch/<user>/nextflow_work/greml_production sbatch run_greml.sh ...
```

The ‘NXF_WORK_ROOT‘ override is strongly recommended for anyone other than the original owner. It prevents Nextflow from trying to write into ‘/gpfs/scratch/sl8085‘, avoids permission problems, and keeps cache cleanup under the current analyst's control.

Run/reuse/resume decision tree:

* If only plotting code changed, do not rerun Nextflow. Rerun the relevant script in ‘scripts/‘.
* If only the GCTA command or GREML parsing changed, it can be safe to reuse both phenotype and GRM/PCA files.
* If expression source, normalization, expression filters, PEER settings, PC count, or sample set changed, do not reuse phenotype files.
* If genotype prefix, SNP set, HM3/no−MHC setting, or unrelated keep list changed, do not reuse GRM/PCA files.
* If using a new unrelated keep file, do not use full ‘−−reuse_run_dir‘ unless the reused phenotype and GRM were already created from that exact keep file.
* If a previous run directory has confusing historical/hash naming, use a fresh ‘GREML_COMBO_LABEL‘ or archive the old run before rerunning.

Reuse both phenotype and GRM/PCA from a previous run:

```
GREML_FANOUT=0 \
NXF_WORK_ROOT=/gpfs/scratch/$USER/nextflow_work/greml_production \
  sbatch run_greml.sh \
  --use_hm3_no_hla false \
  --expression_source tmm \
  --normalization_type raw \
  --reuse_run_dir /gpfs/data/mostafavilab/sool/analysis/GeneExpression/20260428_GE_GEUVADIS_v2/GeneExpre
ssion/runs/all_snps_tmm_raw_peerauto_pmg0_npc5
```

Reuse phenotype only and regenerate GRM/PCA:

```
GREML_FANOUT=0 \
NXF_WORK_ROOT=/gpfs/scratch/$USER/nextflow_work/greml_production \
  sbatch run_greml.sh \
  --use_hm3_no_hla false \
  --expression_source tmm \
  --normalization_type raw \
  --reuse_pheno_dir /path/to/previous/results/data
```

Reuse GRM/PCA only and regenerate phenotypes:

```
GREML_FANOUT=0 \
NXF_WORK_ROOT=/gpfs/scratch/$USER/nextflow_work/greml_production \
  sbatch run_greml.sh \
  --use_hm3_no_hla false \
  --expression_source tmm \
  --normalization_type raw \
  --reuse_grm_dir /path/to/previous/results/pca/all_snps_tmm_raw
```

10. Expected Run Directory Layout

A run directory looks like:

```
runs/all_snps_tmm_raw_peerauto_pmg0_npc5/
  nextflow_logs/
  results/
    data/
    pca/
    summary/
```

Data folder:

```
results/data/pheno_all_snps_tmm_raw.phenotypes.tsv
results/data/pheno_all_snps_tmm_raw.phenotypes.tsv.gz
results/data/pheno_all_snps_tmm_raw.qcovar
results/data/pheno_all_snps_tmm_raw.gene_index_map.txt
results/data/pheno_all_snps_tmm_raw.filtered_gene_ids.txt
```

PCA/GRM folder:

```
results/pca/all_snps_tmm_raw/genotype_grm.grm.bin
results/pca/all_snps_tmm_raw/genotype_grm.grm.N.bin
results/pca/all_snps_tmm_raw/genotype_grm.grm.id
results/pca/all_snps_tmm_raw/geno_pca.eigenvec
```

Summary folder:

```
results/summary/final_heritability_summary_all_snps_tmm_raw.tsv
```

A run is complete enough for downstream plots if it has:

```
results/summary/final_heritability_summary_<runLabel>.tsv
results/data/pheno_<runLabel>.phenotypes.tsv
results/data/pheno_<runLabel>.qcovar
results/data/pheno_<runLabel>.gene_index_map.txt
results/pca/<runLabel>/genotype_grm.grm.bin
results/pca/<runLabel>/genotype_grm.grm.N.bin
results/pca/<runLabel>/genotype_grm.grm.id
results/pca/<runLabel>/geno_pca.eigenvec
nextflow_logs/nextflow_<jobid>.log
```

Optional but useful:

```
results/data/pheno_<runLabel>.filtered_gene_ids.txt
results/data/pheno_<runLabel>.phenotypes.tsv.gz
```

Important file schemas:

```
eur_keep.txt:
  FID    IID

eur_map.tsv:
  RNA_ID    IID    FID

PLINK .fam:
  FID    IID    father    mother    sex    phenotype

pheno_<runLabel>.phenotypes.tsv:
  no header
  FID    IID    gene_1    gene_2    ...    gene_n

pheno_<runLabel>.qcovar:
  no header
  FID    IID    PC1    ...    PC5    PEER1    ...    PEERk

pheno_<runLabel>.gene_index_map.txt:
  gene_name    mpheno_index

final_heritability_summary_<runLabel>.tsv:
  Gene    Status    Index    h2_GREML    SE_GREML    Pval_GREML    h2_HE    SE_HE
```

GCTA−specific detail:

* The phenotype and qcovar files intentionally have no header.
* The gene index map and final summary intentionally do have headers.
* ‘mpheno_index‘ is one−based and refers to the gene columns after FID/IID.

11. Quality Checks After Running

Check final summary exists:

```
ls runs/*_peerauto_pmg0_npc5/results/summary/final_heritability_summary*.tsv
```

Check PASS/FAIL counts:

```
awk −F'\t' 'NR>1 {n[$2]++} END{for (k in n) print k,n[k]}' \
  runs/all_snps_tmm_raw_peerauto_pmg0_npc5/results/summary/final_heritability_summary*.tsv
```

Check sample count in GRM:

```
wc −l runs/all_snps_tmm_raw_peerauto_pmg0_npc5/results/pca/all_snps_tmm_raw/genotype_grm.grm.id
```

Check samples against PLINK ‘.fam‘:

```
PLINK_PREFIX=/gpfs/data/mostafavilab/shared_data/LDSC_data/1000G_EUR_Phase3_plink/1000G.EUR.QC.22
RUN_DIR=/gpfs/data/mostafavilab/sool/analysis/GeneExpression/20260428_GE_GEUVADIS_v2/GeneExpression/runs
/all_snps_tmm_raw_peerauto_pmg0_npc5
comm −23 \
  <(awk '{print $2}' "$RUN_DIR/results/pca/all_snps_tmm_raw/genotype_grm.grm.id" | sort −u) \
  <(awk '{print $2}' "${PLINK_PREFIX}.fam" | sort −u)
```

No output means all used samples are in the genotype panel.

Check that phenotype/covariate files exist:

```
ls −lh runs/all_snps_tmm_raw_peerauto_pmg0_npc5/results/data/pheno_all_snps_tmm_raw.*
```

Check that the phenotype and gene map dimensions are consistent:

```
RUN_DIR=runs/all_snps_tmm_raw_peerauto_pmg0_npc5
PHENO=$RUN_DIR/results/data/pheno_all_snps_tmm_raw.phenotypes.tsv
MAP=$RUN_DIR/results/data/pheno_all_snps_tmm_raw.gene_index_map.txt
awk −F'\t' 'NR==1 {print "phenotype_gene_columns=" NF−2}' "$PHENO"
awk −F'\t' 'NR>1 {n++} END{print "gene_map_rows=" n}' "$MAP"
```

These two numbers should match.

Check the Nextflow log for errors:

```
ls −lt runs/all_snps_tmm_raw_peerauto_pmg0_npc5/nextflow_logs/
tail −n 200 runs/all_snps_tmm_raw_peerauto_pmg0_npc5/nextflow_logs/nextflow_*.log
```

Check whether many genes failed:

```
awk −F'\t' 'NR>1 && $2=="FAIL"{n++} END{print "FAIL_genes=" n+0}' \
  runs/all_snps_tmm_raw_peerauto_pmg0_npc5/results/summary/final_heritability_summary_all_snps_tmm_raw.t
sv
```

Interpretation:

* A small number of failed genes can occur because per−gene GREML models may be unstable.
* A large systematic failure usually indicates an input, environment, GCTA, or covariate problem.

12. Failure Modes And How To Debug

This section follows the same handoff logic as the Salmon workflow document: start from the symptom, identify the likely cause, and then inspect the smallest file that proves what happened.

‘r_peer‘ or R package failure

Symptom:

```
there is no package called 'peer'
there is no package called 'edgeR'
source activate r_peer fails
```

Likely cause:

* The original owner's conda environment is not accessible to the new user.
* The environment exists but does not contain the required R packages.
* The compute node does not inherit the same module/conda setup as the login node.

Check:

```
module load anaconda3/gpu/new
source activate r_peer
Rscript −e 'library(data.table); library(edgeR); library(peer); library(matrixStats); sessionInfo()'
```

Recommended response:

* Create a personal ‘r_peer‘ environment using the commands in the input−file inventory section.
* Do not edit the workflow to remove PEER just to make the environment error disappear. PEER factors are part of the intended covariate model.

Scratch permission or quota failure

Symptom:

```
Permission denied
No space left on device
Failed to create work directory
```

Likely cause:

* A new user is accidentally writing to ‘/gpfs/scratch/sl8085‘.
* The scratch directory does not exist.
* The scratch quota or file−count quota is exhausted.

Check:

```
echo "$USER"
ls −ld /gpfs/scratch/$USER
df −h /gpfs/scratch/$USER
```

Recommended response:

```
mkdir −p /gpfs/scratch/$USER/nextflow_work/greml_production
NXF_WORK_ROOT=/gpfs/scratch/$USER/nextflow_work/greml_production sbatch run_greml.sh
```

FAST phenotype preparation takes many hours

Symptom:

```
PREPARE_PHENOTYPES runs for a long time
```

Likely cause:

* This is expected for full phenotype preparation because the script performs expression filtering, TMM/TPM selection, normalization, and PEER factor estimation across thousands of genes.

Recommended response:

* If phenotype files already exist and the sample set/expression/normalization/PEER settings did not change, reuse them with ‘−−reuse_run_dir‘ or ‘−−reuse_pheno_dir‘.
* If the sample set, unrelated keep list, expression source, normalization, PC count, or PEER settings changed, do not reuse old phenotype files.

Reuse mode cannot find phenotype files

Symptom:

```
Not found: reused phenotype file
Not found: reused legacy phenotype file
```

Likely cause:

* ‘reuse_run_dir‘ points to the wrong run directory.
* The requested run label does not match the phenotype prefix inside ‘results/data‘.
* The run directory uses ‘pheno_<snp>_<source>_<normalization>‘ naming, not legacy ‘final.phenotypes.tsv‘.

Check:

```
REUSE_RUN=/path/to/previous/run
find "$REUSE_RUN/results/data" −maxdepth 1 −type f | sort
```

Recommended response:

* Use the run directory that matches the same SNP set, expression source, and normalization type.
* Or pass ‘−−reuse_pheno_dir <previous−run>/results/data‘ explicitly.

Run lock or Nextflow resume collision

Symptom:

```
another launch is active for RUN_KEY
LOCK file exists
Nextflow LOCK collision
```

Likely cause:

* Two jobs are trying to resume the same run directory.
* A previous launch is still active.
* A stale lock remained after a crash.

Check:

```
squeue −u $USER
ls −lah runs/<run−name>/.nextflow/
```

Recommended response:

* Do not delete locks while a job may still be running.
* Wait for the active job to finish, or submit with a different ‘GREML_COMBO_LABEL‘ if intentionally starting a separate run.
* Use ‘GREML_RUN_LOCK_WAIT_SEC=<seconds>‘ if the desired behavior is to wait instead of fail fast.

‘ESTIMATE_HERITABILITY‘ exit status 2 for many genes

Symptom:

```
Process ‘ESTIMATE_HERITABILITY (...)‘ terminated with an error exit status (2)
```

Likely causes:

* GCTA failed before producing a valid ‘.hsq‘ file.
* The GCTA executable or runtime libraries are broken on the compute node.
* Input phenotype/qcovar/GRM files are mismatched.
* HE regression was enabled and hit the known MKL runtime problem.

Find one failing work directory:

```
WORK_ROOT=/gpfs/scratch/$USER/nextflow_work/greml_production/<run−name>
find "$WORK_ROOT" −name .exitcode −exec sh −c 'c=$(cat "$1"); [ "$c" != 0 ] && echo "$(dirname "$1") $c"
' sh {} \; | head
```

If the Nextflow run name is known, also query the Nextflow task table:

```
NXF_LOG=runs/<run−name>/nextflow_logs/nextflow_<jobid>.log
RUN_NAME=<nextflow_run_name>
nextflow −log "$NXF_LOG" log "$RUN_NAME" −f 'hash,name,status,exit,native_id,workdir' | grep ESTIMATE_HE
RITABILITY | head
```

Inspect it:

```
WD=/path/to/failing/workdir
cat "$WD/.command.sh"
cat "$WD/.command.err"
tail −n 120 "$WD"/*_greml.gcta.log
ls −lh "$WD"
```

Recommended response:

* If the GCTA log says missing MKL libraries during HE regression, keep ‘run_he=false‘.
* If the GCTA log says phenotype or qcovar samples do not match the GRM, rebuild phenotype and GRM from the same sample set.
* If the GCTA log is empty and Slurm killed the job, check ‘sacct‘ for memory/time/node failure.

External termination or scheduler kill

Symptom:

```
terminated for an unknown reason −− likely terminated by the external system
```

Likely cause:

* Slurm killed the task because of time, memory, preemption, node failure, or administrative scheduling pressure.

Check:

```
sacct −j <slurm_job_id> −−format=JobID,JobName%40,State,ExitCode,Elapsed,ReqMem,MaxRSS,NodeList,Reason −
P
```

Recommended response:

* If the state is ‘OUT_OF_MEMORY‘, increase memory for ‘ESTIMATE_HERITABILITY‘.
* If the state is ‘TIMEOUT‘, increase time.
* If the state is node failure/preemption, resume the workflow.
* The workflow retries common external−kill statuses, but repeated kills should be investigated rather than ignored.

‘SUMMARIZE_RESULTS‘ syntax or empty filename failure

Symptom:

```
syntax error near unexpected token ‘newline'
echo ... >
cat *.stats >>
```

Likely cause:

* ‘summary_filename‘ resolved to an empty string.
* This was previously observed and fixed by forcing a fallback filename in ‘SUMMARIZE_RESULTS‘.

Check:

```
grep −n "summary_filename" nf/main.nf nf/nextflow.config
```

Recommended response:

* Make sure the current ‘nf/main.nf‘ includes the fallback logic in ‘SUMMARIZE_RESULTS‘.
* Avoid passing ‘−−summary_filename ""‘.

Published file or project directory file−count failure

Symptom:

```
Failed to publish file
```

Likely cause:

* The lab project directory hit file−count or storage limits.
* Too many diagnostics were being published.

Recommended response:

* Keep high−volume intermediate files in scratch.
* Publish only stable final outputs under ‘results/‘.
* Do not publish one diagnostic file per gene unless there is a specific debugging need.

Unexpectedly tiny unrelated sample set

Symptom:

```
Kept after −−grm−cutoff 0.025: very small number
```

Likely cause:

* GCTA ‘−−grm−cutoff 0.025‘ is too strict for this expression−analysis context if applied after constructing a GRM from closely related samples.
* The intended Hernandez−style behavior is to provide an external unrelated keep list, not necessarily to use ‘−−grm−cutoff 0.025‘ inside this workflow.

Recommended response:

* Use a curated 1000 Genomes unrelated keep file through ‘−−unrelated_keep_file‘.
* Confirm the resulting sample count in ‘genotype_grm.grm.id‘.

Many genes have ‘Status=FAIL‘

Symptom:

```
final_heritability_summary_*.tsv contains many FAIL rows
```

Likely causes:

* Per−gene GCTA convergence or boundary issues.
* Input mismatch.
* GCTA runtime problem.
* Scheduler kills.

Recommended response:

* First count FAIL rows by run.
* Then inspect several failing work directories and GCTA logs.
* Do not interpret a run's h2 distribution until the failure mechanism is understood.

13. Downstream Analysis Scripts And Plots

All diagnostic analyses write under:

```
runs/_analysis/
```

Global h2 summary boxplot

Script:

```
scripts/plot_greml_h2_summary_boxplot.R
```

Outputs:

```
runs/_analysis/plot_greml_h2_summary_boxplot/tables/summary_of_summaries.tsv
runs/_analysis/plot_greml_h2_summary_boxplot/plots/greml_h2_boxplot_by_setting.png
```

Scientific rationale:

* Shows distribution of PASS ‘h2_GREML‘ estimates across all settings.
* Helps identify which preprocessing choices inflate, deflate, or destabilize h2.
* Mean labels are useful because h2 distributions can be skewed.

Pairwise h2 comparisons

Script:

```
scripts/pairwise_h2_comparisons.R
```

Main plot categories:

```
SNP set: ALL vs HM3/no-MHC
Expression source: TMM vs TPM
Normalization: RAW vs IRNT
```

Scientific rationale:

* Each scatter plot compares gene-by-gene h2 estimates between two settings.
* Identity line shows perfect agreement.
* Color by expression level tests whether low expression drives disagreement.
* Color by Wald Z tests whether discrepancies are concentrated among weak or strong h2 estimates.
* Discrepancy counts quantify genes where one method gives high h2 and another gives near-zero h2.

Important plot defaults:

```
expression color caps: log2(mean expression + 1) capped at 1.5, 5, or 8 depending on plot
skewness color caps: skewness capped at 1, 2, or 5 depending on plot
TPM vs TMM comparison: TMM is plotted on x-axis for TPM/TMM h2 comparisons
```

Expression decile h2 analysis

Script:

```
scripts/tmm_expression_h2_deciles.R
```

Outputs:

```
runs/_analysis/tmm_expression_h2_deciles/plots/tmm_expression_h2_deciles_boxplot.png
runs/_analysis/tmm_expression_h2_deciles/plots/median_tpm_expression_h2_deciles_boxplot.png
runs/_analysis/tmm_expression_h2_deciles/plots/expression_skewness_by_decile_boxplot.png
```

Scientific rationale:

* Tests whether gene-level h2 is higher for more highly expressed genes.
* Uses TMM RAW mean expression and TPM RAW median expression as abundance bins.
* Filters very low expression genes to avoid bins dominated by near-zero signals.
* Adds skewness by expression bin to check whether distribution shape changes across abundance strata.

Default thresholds:

```
MIN_LOG2_MEAN_TMM_PLUS1 = 1.5
MIN_LOG2_MEDIAN_TPM_PLUS1 = 0
```

Interpretation:

* The low-expression cutoff removes genes whose apparent low h2 may be driven by near-zero expression rather than biology.
* Decile plots answer whether more highly expressed genes have higher or more stable h2 estimates.

TMM raw skewness analysis

Script:

```
scripts/tmm_raw_skewness_analysis.R
```

Outputs include:

```
tmm_raw_skewness_hist_density_<METHOD>.png
tmm_raw_skewness_mean_vs_skew_<METHOD>.png
tmm_raw_skewness_selected_gene_distributions_<METHOD>.png
tmm_vs_tpm_h2_discrepancy_expr_distributions_<METHOD>.png
all_raw_h2_vs_hm3_raw_h2_tmm_colored_by_log2meanexpr_cap*.png
all_raw_h2_vs_hm3_raw_h2_tmm_colored_by_skewness_cap*.png
```

Scientific rationale:

* Skewness detects whether raw expression phenotypes are dominated by long right tails or zero inflation.

* Selected gene histograms give concrete examples for lab discussion.
* H2 discrepancy examples test whether TPM/TMM disagreement is visible at the expression distribution level.

* ALL−vs−HM3 colored scatter tests whether SNP−set sensitivity is linked to expression abundance or skewness.

Important plot defaults:

```
skewness histogram bins: high−resolution defaults in the script
non−zero−inflated high−skew examples: selected to avoid examples driven only by near−zero counts
strict high−count examples: includes genes where most counts are > 10
color caps for ALL−vs−HM3 h2 plots: expression caps 1.5, 5, 8 and skewness caps 1, 2, 5
```

TPM vs TMM phenotype correlations

Script:

```
scripts/tpm_tmm_peer_correlations.R
```

Outputs:

```
tpm_tmm_peer_gene_correlations.tsv
tpm_tmm_peer_correlation_summary.tsv
tpm_tmm_peer_correlation_boxplot.png
tpm_tmm_peer_mean_scatter.png
```

Scientific rationale:

* Compares TPM−derived and TMM−derived phenotype values directly.
* High per−gene correlation means expression source choice is unlikely to change that gene's phenotype ranking across individuals.

* Low correlation identifies genes where TPM/TMM choice may alter h2.

Regulatory/repeat downstream analysis

Script:

```
scripts/downstream_h2_regulatory_repeat_analysis.R
```

This is a separate biological follow−up pipeline. It links h2 to:

* ‘s_het post_mean‘
* gene regulatory burden
* enhancer/open chromatin overlap
* repeat classes/families/subtypes
* randomized repeat background

Scientific rationale:

* Tests whether gene expression heritability varies with evolutionary constraint.
* Tests whether regulatory architecture or repeat burden differs across gene bins.
* Observed−vs−background repeat analysis asks whether repeat patterns exceed

random genomic expectation.

14. Interpretation Guide For Main Plots

h2 boxplot by setting

Use this first. It answers:

* Which pipeline choices produce higher h2?
* Are distributions broad, compressed, or outlier−heavy?
* Are PASS counts similar across methods?

Do not interpret differences without checking PASS/FAIL counts.

Pairwise h2 scatter plots

Use these to diagnose gene−level stability.

Interpretation:

* Points on diagonal: stable gene h2 across methods.
* Points above diagonal: y−axis method estimates higher h2.
* Points below diagonal: x−axis method estimates higher h2.
* Low−expression colored outliers: likely expression−scale or low−information issue.
* High−Z outliers: more likely biologically meaningful or at least statistically stable.

Expression decile h2 plots

Use these to ask whether expression abundance drives h2.

Interpretation:

* Increasing h2 across deciles: low−expression genes may suppress h2 estimates.
* Flat h2 across deciles: h2 estimates are robust to abundance after filtering.
* Mean greater than median: h2 distribution is right−skewed in that bin.

Skewness plots

Use these to ask whether raw expression distributions are suitable for GREML.

Interpretation:

* High positive skew: a few individuals have much higher expression.
* High zero fraction: raw−scale h2 may be unstable.
* IRNT should reduce sensitivity to skew because it rank−normalizes per gene.

TPM/TMM correlation plots

Use these to ask whether expression source changes phenotype rankings.

Interpretation:

* High correlation: TPM and TMM mostly preserve individual−level ordering.
* Low correlation: TPM/TMM choice can change the phenotype and therefore h2.

15. What This Workflow Does Not Do

* It does not start from FASTQ files. FASTQ−to−matrix processing belongs to the GEUVADIS Salmon workflow.
* It does not run read−level RNA QC, FastQC, MultiQC, adapter trimming, or alignment QC.
* It does not automatically validate whether the configured genotype prefix is genome−wide or chromosome−specific.
* It does not automatically derive a Hernandez exact unrelated keep set. A keep file must be provided explicitly.
* It does not publish every per−gene GCTA log by default. High−volume per−gene artifacts remain in scratch work directories.
* It does not guarantee every gene passes GREML just because Nextflow completes.
* It does not estimate total gene expression heritability from all possible variant classes. It estimates variance captured by the GRM built from the selected SNP set.
* It does not make TPM, TMM, RAW, and IRNT estimates directly interchangeable. These are different phenotype definitions.

16. Known Caveats

1. The default genotype prefix ends with ‘.22‘. Confirm whether the analysis is intentionally chromosome 22 only or whether genome−wide genotype files should be merged or supplied.

2. ‘params.grm_prefix‘ exists but the code currently uses hard−coded ‘genotype_grm‘.

3. ‘run_he=true‘ may fail unless the GCTA AppRun MKL runtime issue is fixed. The current workflow intentionally sets ‘run_he=false‘.

4. Full reuse mode skips sample filtering and phenotype preparation. If ‘unrelated_keep_file‘ is set while reusing GRM and phenotype files, the keep file does not change those reused artifacts.

5. Phenotype−only reuse with a newly generated GRM can mismatch normalization and sample set. The workflow warns but does not stop.

6. Per−gene GREML failures do not necessarily fail the whole workflow. Always inspect ‘Status‘ in the summary table.

7. Several downstream scripts have hard−coded HPC paths. Before rerunning on a new project copy, inspect the first 20−40 lines of each R script.

8. Plot files are generated on the HPC run directory under ‘runs/_analysis/‘; they are not stored in this local repo by default.

17. Cleanup And Archiving

The durable scientific outputs are in ‘results/‘ and ‘nextflow_logs/‘. The high−volume task cache is in scratch. Treat these differently.

Archive phenotype files before cleanup:

```
scripts/zip_phenofiles.sh \
  --data-dir runs/all_snps_tmm_raw_peerauto_pmg0_npc5/results/data
```

Dry−run cleanup for one run:

```
scripts/cleanup_nextflow_cache.sh \
  --run-dir runs/all_snps_tmm_raw_peerauto_pmg0_npc5 \
  --nxf-work /gpfs/scratch/$USER/nextflow_work/greml_production/all_snps_tmm_raw_peerauto_pmg0_npc5
```

Execute cleanup of cache/log/work files:

```
scripts/cleanup_nextflow_cache.sh \
  --run-dir runs/all_snps_tmm_raw_peerauto_pmg0_npc5 \
  --nxf-work /gpfs/scratch/$USER/nextflow_work/greml_production/all_snps_tmm_raw_peerauto_pmg0_npc5 \
  --execute
```

Important warnings:

* Do not use cleanup options that delete ‘results/‘ unless you intentionally want to remove durable outputs.
* Keep ‘results/summary‘, ‘results/data‘, ‘results/pca‘, and ‘nextflow_logs‘ for reproducibility.
* Once scratch cache is deleted, Nextflow ‘−resume‘ cannot reuse those per−task work directories.
* If storage pressure is the problem, archive first, then clean scratch; do not silently delete the only copy of phenotype or summary files.

18. Minimal Handoff Checklist

Before a new analyst reruns the workflow:

1. Confirm the genotype prefix is correct and genome−wide if needed.
2. Confirm the GEUVADIS TPM/count matrices exist.
3. Confirm ‘gcta_path‘ works on compute nodes.
4. Check whether the original ‘r_peer‘ conda environment is accessible.
5. If not, create a personal ‘r_peer‘ replacement and confirm ‘data.table‘, ‘edgeR‘, ‘peer‘, and ‘matrixStats‘ load in R.
6. Create a personal scratch root such as ‘/gpfs/scratch/$USER/nextflow_work/greml_production‘.
7. Submit with ‘NXF_WORK_ROOT=/gpfs/scratch/$USER/nextflow_work/greml_production‘.
8. Decide whether to provide ‘unrelated_keep_file‘.
9. Decide whether to run all 12 settings or a single setting.
10. Keep ‘run_he=false‘ unless the MKL issue is solved.
11. Run a single small/default job first if changing input paths.
12. Check sample count in ‘genotype_grm.grm.id‘.
13. Check PASS/FAIL counts in the final summary.
14. Run diagnostic plots under ‘scripts/‘.
15. Archive both ‘nextflow_logs/‘ and ‘results/‘.

19. Running A Simple New GREML Analysis

This repository can be adapted to a new expression heritability analysis, but do not treat it as a one−line path swap. The key question is whether the new data still satisfy the same contract as the GEUVADIS/1000G run.

If using new expression matrices with the same GEUVADIS/1000G sample IDs:

* Update ‘params.tpm_file‘.
* Update ‘params.counts_file‘.
* Confirm both matrices have the same genes and sample columns.
* Confirm sample columns can still be mapped through the GEUVADIS SDRF ‘ENA_RUN‘ IDs.
* Run one single setting before launching fanout.

If using a new genotype panel:

* Update ‘params.plink_prefix‘.
* Confirm ‘.bed‘, ‘.bim‘, and ‘.fam‘ exist.
* Confirm FAM IIDs can be matched to the expression sample IDs.
* Confirm the HM3/no−MHC SNP list uses SNP IDs that exist in the new ‘.bim‘, or disable HM3 mode.
* Rebuild GRM/PCA; do not reuse old GRM files.

If using a new sample set:

* Provide a new metadata/map strategy or rewrite ‘FILTER_EUROPEANS‘.
* Do not reuse phenotype files from the old sample set.

* Do not reuse GRM/PCA from the old sample set.
* Reconsider PEER factor count because the GTEx−style rule depends on sample size.

If only changing downstream plots:

* Do not rerun Nextflow.
* Rerun only the relevant script in ‘scripts/‘.
* Keep the same ‘runs/‘ directory as input and write new plots under ‘runs/_analysis‘.

Smoke−test command for one setting:

```
cd /gpfs/data/mostafavilab/sool/analysis/GeneExpression/20260428_GE_GEUVADIS_v2/GeneExpression
mkdir −p /gpfs/scratch/$USER/nextflow_work/greml_production
GREML_FANOUT=0 GREML_RESUME=0 \
NXF_WORK_ROOT=/gpfs/scratch/$USER/nextflow_work/greml_production \
  sbatch run_greml.sh \
  −−use_hm3_no_hla false \
  −−expression_source tmm \
  −−normalization_type raw
```

Current limitation:

* The workflow does not currently expose a ‘max_genes‘ or small−gene test parameter.
* A true small test mode would need to be added deliberately, for example by adding a parameter that subsets the gene index map before ‘ESTIMATE_HERITABILITY‘.
* Do not manually edit production phenotype files just to make a small test unless the output directory is clearly marked as a temporary development run.

20. Source List

GEUVADIS/1000G:

* International Genome Sample Resource GEUVADIS data collection:
https://www.internationalgenome.org/data−portal/data−collections/geuvadis.html

* GEUVADIS project summary: RNA−seq from 1000 Genomes samples across CEU, FIN, GBR, TSI, and YRI.

Salmon:

* Patro et al. Salmon provides fast and bias−aware quantification of transcript expression. Nature Methods 2017. https://www.nature.com/articles/nmeth.4197

TMM:

* Robinson and Oshlack. A scaling normalization method for differential expression analysis of RNA−seq data. Genome Biology 2010.
https://genomebiology.biomedcentral.com/articles/10.1186/gb−2010−11−3−r25

GTEx methods:

* GTEx Analysis Methods V7 PDF.
https://storage.googleapis.com/gtex−public−data/Portal_Analysis_Methods_v7_09052017.pdf

PEER environment:

* Bioconda ‘peer‘ package page.
https://anaconda.org/bioconda/peer

* Bioconda ‘r−peer‘ files page.
https://anaconda.org/bioconda/r−peer/files

GCTA/GREML:

* Yang et al. GCTA: A Tool for Genome−wide Complex Trait Analysis. American

Journal of Human Genetics 2011. https://www.sciencedirect.com/science/article/pii/S0002929710005987

* GCTA documentation and notes on GRM and relatedness pruning:
https://yanglab.westlake.edu.cn/software/gcta/

GEUVADIS expression heritability:

* Hernandez et al. Ultrarare variants drive substantial cis heritability of human
gene expression. Nature Genetics 2019. https://www.nature.com/articles/s41588−019−0487−7

* Dahl et al. On Negative Heritability and Negative Estimates of Heritability.
Genetics 2020. https://academic.oup.com/genetics/article/215/2/343/5930411